

Data Elegance in Bits: Exploring and Implementing Huffman Coding and Lempel–Ziv–Welch (LZW) Compression Algorithms

A PROJECT REPORT

Submitted by

Gagnesh Kakkar(23BCS11196)

in partial fulfilment for the award of the degree of

Bachelors of Engineering

IN

Computer Science Engineering



TABLE OF CONTENTS

Overview.....	3
CHAPTER 1. INTRODUCTION.....	4
1.1 Introduction.....	4
CHAPTER 2. THEORY OF OPERATION.....	5
2.1 Huffman Coding	5
2.2 LZW (Lempel–Ziv–Welch) Compression.....	6
CHAPTER 3. FILE FORMAT & DESIGN DECISIONS.....	7
3.1 File Format & Design Decisions.....	7
CHAPTER 4. CODE IMPLEMENTATION & ANALYSIS.....	8
4.1 Implementation Overview (C++).....	8
4.1 Complexity and Resource Analysis.....	8
4.1 Validation and Testing	9
CHAPTER 5. CONCLUSION AND FUTURE WORK.....	10
5.1 Conclusion.....	10
5.2 Limitations and Future Enhancements.....	10
APPENDIX.....	13
Appendix A: Build & Usage.....	13
Appendix B: Source Code (Huffman, LZW, CLI).....	13

OVERVIEW

Title of Project:

Data Elegance in Bits: Exploring and Implementing Huffman Coding and Lempel–Ziv–Welch (LZW) Compression Algorithms

DOMAIN: Data Compression, Information Theory

Description:

This project explores two classic lossless data compression algorithms—Huffman coding and Lempel–Ziv–Welch (LZW)—and implements production-style reference code in modern C++. We study their theory (prefix codes vs. dictionary-based compression), analyze complexity, discuss design tradeoffs (speed, memory, determinism), and provide a testable command-line tool that can compress and decompress arbitrary files with either algorithm.

Hardware / Software Requirement:

- CPU: Any modern x64/ARM (single core is sufficient for demos)
- RAM: ≥ 1 GB recommended (both algorithms work with far less for small files)
- OS: Windows / Linux / macOS
- Toolchain: C++17 compiler (GCC/Clang/MSVC) and CMake ≥ 3.10
- Optional libraries: None beyond the standard library

Technology:

- Huffman coding (entropy codes with variable-length prefix codes)
- LZW dictionary compression (fixed 12-bit codes, max 4096 entries)
- Bit-level packing, file I/O, portable little-endian headers
- Build system: CMake

Objective:

Develop methods to save storage space and transmit data more efficiently by implementing, evaluating, and comparing two core lossless compression approaches: entropy coding (Huffman) and dictionary coding (LZW).

Targeted Outcome:

- Smaller file sizes for storage

- Faster or more efficient data transmission
- Reproducible, portable reference implementations with clear interfaces

CHAPTER 1.

INTRODUCTION

Data compression plays a crucial role in modern computing, particularly as the volume of digital information continues to grow. Lossless compression is a category of compression techniques designed to reduce data size without losing any part of the original information. When the data is decompressed, it is restored exactly to its original form. This is essential in scenarios where accuracy cannot be compromised, such as text files, source code, executable programs, medical data, and configuration files.

This project focuses on two of the most influential and widely used lossless compression algorithms:

- **Huffman Coding:**
This algorithm assigns variable-length bit patterns to symbols based on their frequencies in the input data. Symbols that occur more frequently are assigned shorter codes, while less frequent symbols receive longer codes. This leads to efficient bit-level storage with minimal redundancy.
- **Lempel–Ziv–Welch (LZW) Compression:**
LZW builds a dictionary of repeated sequences encountered in the input. Instead of storing repeated patterns multiple times, LZW replaces them with dictionary indices, resulting in compact representation and reduced overall data size.

Both algorithms form the conceptual foundation of many modern compression standards. For example:

- The DEFLATE algorithm (used in ZIP, PNG, and GZIP) uses Huffman coding as part of its compression stage.
- The GIF image format uses LZW as its primary compression technique.

These algorithms are especially effective on files where patterns or character frequency distributions are uneven, such as:

- Log files
- CSV datasets
- Programming source code

- Plain text documents

However, applying these methods to already-compressed multimedia formats like JPEG images, MP3 audio, MP4 video, or ZIP archives rarely yields further compression, because such formats already remove redundancy internally.

CHAPTER 2.

THEORY OF OPERATION

2.1 Huffman Coding

Huffman coding is based on the idea of assigning bit patterns to symbols proportional to their likelihood of appearing. It constructs a binary tree where each leaf node corresponds to a byte value (0–255).

Process:

1. **Frequency Counting:**
Determine how often each byte value appears in the data.
2. **Tree Construction:**
Combine the least frequent symbols into subtrees in a repeated merging process, forming a full Huffman tree.
3. **Code Generation:**
Assign binary digits: moving left = 0 and right = 1 in the tree.
A symbol's code is derived from the path to its leaf node.
4. **Encoding:**
Replace each input byte with its corresponding bit sequence.
5. **Decoding:**
Reverse the process by traversing the tree according to incoming bits.

Advantages:

- Produces optimal prefix-free codes for the given symbol frequency distribution.
- Ensures no code is a prefix of another, preventing ambiguity in decoding.

Computational Complexity:

- Tree Construction: $O(n \log \sigma)$ where $\sigma \leq 256$ (size of byte alphabet)
- Encoding and Decoding: $O(n)$

2.2 LZW (Lempel–Ziv–Welch) Compression

Unlike Huffman coding, LZW processes data by identifying repeated sequences rather than symbol frequencies.

Process (Encoder):

1. Start with a dictionary containing all possible single-byte values (0–255).
2. Read characters from the input and gradually build the longest sequence found in the dictionary.
3. When a sequence is no longer found, output its dictionary index.
4. Add the new sequence to the dictionary.
5. Repeat until the entire input is processed.

The dictionary can grow up to 4096 entries, corresponding to 12-bit code words.

Decoder:

- Reconstructs the same dictionary during decoding.
- Must carefully handle the special case where the decoder encounters a code that has not yet been added (known as the *KwKwK* situation).

Performance:

- Works particularly well when patterns repeat, such as in programming source files, markup, and uncompressed text.

CHAPTER 3.

FILE FORMAT & DESIGN DECISIONS

To ensure compatibility and simplify decompression, a custom wrapper format is used:

Field	Description
magic	4 bytes: "CMP1" identifies the file as compressed
algo	1 byte: 0x01 for Huffman, 0x02 for LZW
orig_size	8-byte little-endian integer indicating the size before compression
payload	Encoded data (algorithm-specific format)

Huffman Payload

- Frequency table (256×32 -bit integers)
- Bit-packed compressed stream (LSB-first)

LZW Payload

- Sequence of 12-bit dictionary codes, packed tightly into bytes

This format ensures that:

- Decompression does **not** require external metadata.
- Files remain self-describing and portable across systems.

CHAPTER 4.

CODE Implementation & Analysis

4.1 Implementation Overview (C++):

Source File Roles:

File	Purpose
Huffman.h / Huffman.cpp	Tree construction, code generation, bit-level encoding and decoding
LZW.h / LZW.cpp	Dictionary-based compression and decompression
main.cpp	Command-line interface for selecting algorithm and mode
CMakeLists.txt	Cross-platform build system configuration

Key Design Strengths:

- Uses only the C++ Standard Library (portable).
- Includes explicit error handling for corrupted data.
- Handles edge cases such as empty inputs and single-symbol inputs.

4.2 Complexity and Resource Analysis

Algorithm	Build Complexity	Compression/Decompression Complexity	Memory Usage
Huffman Coding	$O(\sigma \log \sigma)$, $\sigma \leq 256$	$O(n)$	Tree + frequency table
LZW (12-bit)	Dictionary grows dynamically	$O(n)$ average	Dictionary of ≤ 4096 entries

Huffman performs best when the input has uneven character distribution, while LZW excels when patterns repeat.

4.3 Validation and Testing

Testing is performed to ensure:

- Correctness: Output after decompression exactly matches original input.
- Compression efficiency: Compare compression ratio across file types.
- Platform consistency: Encoding and decoding work the same on Windows/Linux.

Example Test Commands:

```
kotlin
```

```
./compress_tool compress huffman data.txt data.huf  
./compress_tool decompress huffman data.huf restored.txt  
diff data.txt restored.txt
```

CHAPTER 5.

CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project demonstrates the design, implementation, and evaluation of two fundamental lossless compression algorithms: **Huffman Coding** and **Lempel–Ziv–Welch (LZW)**. Both algorithms address redundancy in data, but they do so using different strategies—Huffman Coding uses statistical frequency information to assign optimal variable-length codes, while LZW builds a dynamic dictionary to represent repeated sequences.

The implementation in modern C++ shows how these algorithms can be constructed using efficient data structures such as priority queues, hash maps, and binary trees. The project also ensures portability and reliability through a unified file format that enables accurate reconstruction of the original data across different platforms.

The comparative analysis indicates that:

- **Huffman Coding** performs well when symbol frequencies vary noticeably in the input data.
- **LZW** provides significant advantages when data contains repeated patterns or structural redundancy.

Together, these methods form the foundational concepts behind many real-world compression technologies, including ZIP archives, PNG images, network protocols, and document formats. The project not only highlights how lossless compression reduces storage requirements and improves data transfer efficiency but also demonstrates how theoretical concepts are translated into practical software systems.

Overall, the work provides a strong conceptual and practical platform for further study into more advanced compression techniques, hybrid models, and performance-optimized implementations.

5.2 Limitations and Future Enhancements

Limitations

While the implemented Huffman Coding and LZW algorithms successfully demonstrate practical lossless data compression, there are several areas where performance, flexibility, and scalability can be further improved.

1. Fixed Dictionary Size in LZW

The current implementation of LZW uses a fixed 12-bit dictionary, allowing a maximum of 4096 entries. While this ensures predictable memory usage and compatibility with historical implementations like GIF, it also limits the compression capability for large or highly repetitive datasets. When the dictionary becomes full, new patterns are no longer added, reducing compression efficiency over time.

Possible Enhancement:

Implement **adaptive code-width expansion**, where the dictionary begins with 9-bit entries and grows dynamically up to 12 or even 16 bits as new patterns are discovered. This approach maintains efficiency early in the compression process while enabling higher compression ratios for larger inputs.

2. Hybrid Compression Approach

Huffman coding and LZW operate on different principles: Huffman reduces symbol-level redundancy, while LZW compresses repeated sequences. Each works best with specific types of data.

Potential Improvement:

Apply **LZW first** to capture repeated sequences across the dataset, and then apply **Huffman coding** to the resulting output to remove remaining symbol-level redundancy.

This two-stage model is similar to the **DEFLATE** algorithm used in ZIP and PNG, and often produces significantly better compression ratios than either method alone.

3. Parallel and Multithreaded Compression

The current implementation processes input data sequentially, which is efficient for small files but can become slow for large datasets.

Future Enhancement:

Introduce **parallel compression**, where the input file is divided into independent blocks that can be processed simultaneously across multiple CPU cores. After compressing each block, the results can be stored with metadata to allow independent decompression. Modern compression utilities like *Zstandard* and *LZ4* rely heavily on multithreading to achieve high throughput.

4. Streaming Compression and Decompression

The present implementation reads the entire file into memory before processing. This is adequate for small and medium-sized files but becomes problematic for very large datasets or memory-constrained environments (e.g., embedded systems).

Future Enhancement:

Develop a **streaming API** that processes the data incrementally as it arrives (chunk-based compression). This allows:

- Processing of arbitrarily large data sizes
- Lower memory footprint
- Real-time compression for network transmission

Streaming support is valuable in cloud applications, live communication protocols, and large-scale servers.

APPENDIX

Appendix A: Build & Usage

Prerequisites: CMake ≥ 3.10 and a C++17 compiler.

Build:

```
mkdir build && cd build
```

```
cmake ..
```

```
cmake --build . -j
```

Usage:

```
compress_tool compress huffman <input> <output.cmp>
```

```
compress_tool decompress huffman <input.cmp> <output>
```

```
compress_tool compress lzw <input> <output.cmp>
```

```
compress_tool decompress lzw <input.cmp> <output>
```

Appendix B: Source Code (Huffman, LZW, CLI)

(Complete source is included below and in separate .cpp/.h files.)

Huffman.h

```
#pragma once
#include <stdint>
#include <vector>
#include <array>
#include <queue>
#include <memory>
#include <unordered_map>
#include <string>
#include <fstream>

// Simple Huffman codec for arbitrary byte streams.
// File format used by this project when writing compressed data:
// [magic: 'CMP1'] [algo: 1 byte=0x01 for Huffman]
// [orig_size: uint64 little-endian]
```

```

// [freq table: 256 * uint32 little-endian for each byte value]
// [payload: bit-packed codes; padded to full bytes]
//
// Decoding reconstructs the tree from the frequency table and emits
// exactly orig_size bytes by traversing the bitstream.
class Huffman {
public:
    struct Node {
        uint64_t freq;
        int16_t symbol; // -1 for internal node, 0..255 for leaf
        std::unique_ptr<Node> left;
        std::unique_ptr<Node> right;
        Node(uint64_t f, int16_t s) : freq(f), symbol(s) {}
        Node(uint64_t f, std::unique_ptr<Node> l, std::unique_ptr<Node> r)
            : freq(f), symbol(-1), left(std::move(l)), right(std::move(r)) {}
    };

    static void compress(std::istream& in, std::ostream& out);
    static void decompress(std::istream& in, std::ostream& out);

private:
    using CodeMap = std::array<std::vector<bool>, 256>;

    static std::unique_ptr<Node> buildTree(const std::array<uint64_t, 256>& freq);
    static void buildCodes(const Node* node, std::vector<bool>& path, CodeMap& codes);
    static void writeU32(std::ostream& out, uint32_t v);
    static void writeU64(std::ostream& out, uint64_t v);
    static uint32_t readU32(std::istream& in);
    static uint64_t readU64(std::istream& in);
};

```

Huffman.cpp

```

#include "Huffman.h"
#include <stdexcept>
#include <cstring>

namespace {
    struct Cmp {
        bool operator()(const std::unique_ptr<Huffman::Node>& a,
            const std::unique_ptr<Huffman::Node>& b) const {
            return a->freq > b->freq; // min-heap
        }
    };
}

```

```

void Huffman::writeU32(std::ostream& out, uint32_t v) {
    for (int i=0;i<4;++i) out.put(static_cast<char>((v >> (8*i)) & 0xFF));
}
void Huffman::writeU64(std::ostream& out, uint64_t v) {
    for (int i=0;i<8;++i) out.put(static_cast<char>((v >> (8*i)) & 0xFF));
}
uint32_t Huffman::readU32(std::istream& in) {
    uint32_t v=0; for (int i=0;i<4;++i){ int c=in.get(); if(c==EOF) throw
std::runtime_error("Unexpected EOF"); v |= (uint32_t(uint8_t(c)) << (8*i)); }
    return v;
}
uint64_t Huffman::readU64(std::istream& in) {
    uint64_t v=0; for (int i=0;i<8;++i){ int c=in.get(); if(c==EOF) throw
std::runtime_error("Unexpected EOF"); v |= (uint64_t(uint8_t(c)) << (8*i)); }
    return v;
}

std::unique_ptr<Huffman::Node> Huffman::buildTree(const std::array<uint64_t,256>& freq)
{
    std::priority_queue<std::unique_ptr<Node>, std::vector<std::unique_ptr<Node>>, Cmp>
pq;
    for (int i=0;i<256;++i) {
        if (freq[i] > 0) {
            pq.push(std::make_unique<Node>(freq[i], int16_t(i)));
        }
    }
    if (pq.empty()) {
        // empty input; create a dummy node
        return std::make_unique<Node>(1, int16_t(0));
    }
    if (pq.size() == 1) {
        // Edge case: only one symbol. Make a tree with a single child.
        auto lone = std::move(const_cast<std::unique_ptr<Node>&>(pq.top()));
        pq.pop();
        auto parent = std::make_unique<Node>(lone->freq, std::move(lone),
std::make_unique<Node>(0, int16_t((lone->symbol+1)%256)));
        return parent;
    }
    while (pq.size() > 1) {
        auto a = std::move(const_cast<std::unique_ptr<Node>&>(pq.top())); pq.pop();
        auto b = std::move(const_cast<std::unique_ptr<Node>&>(pq.top())); pq.pop();
        auto parent = std::make_unique<Node>(a->freq + b->freq, std::move(a), std::move(b));
        pq.push(std::move(parent));
    }
    auto root = std::move(const_cast<std::unique_ptr<Node>&>(pq.top())); pq.pop();
    return root;
}

```

```

}

void Huffman::buildCodes(const Node* node, std::vector<bool>& path, CodeMap& codes) {
    if (!node) return;
    if (node->symbol >= 0) {
        if (path.empty()) { path.push_back(false); } // single-symbol edge case
        codes[static_cast<uint8_t>(node->symbol)] = path;
        return;
    }
    path.push_back(false);
    buildCodes(node->left.get(), path, codes);
    path.back() = true;
    buildCodes(node->right.get(), path, codes);
    path.pop_back();
}

void Huffman::compress(std::istream& in, std::ostream& out) {
    // Read input and collect bytes
    std::vector<uint8_t> data((std::istreambuf_iterator<char>(in)),
std::istreambuf_iterator<char>());
    uint64_t orig_size = data.size();

    std::array<uint64_t, 256> freq{};
    for (auto b : data) freq[b]++;

    auto root = buildTree(freq);

    CodeMap codes{};
    std::vector<bool> path;
    buildCodes(root.get(), path, codes);

    // Header
    out.write("CMP1", 4);
    out.put(char(0x01)); // algo id: Huffman
    writeU64(out, orig_size);
    for (int i=0; i<256; ++i) writeU32(out, static_cast<uint32_t>(freq[i]));

    // Bit-pack
    uint8_t buffer = 0; int bitcount = 0;
    auto flush_bit = [&](bool bit){
        buffer |= (uint8_t(bit) << bitcount);
        bitcount++;
        if (bitcount == 8) {
            out.put(static_cast<char>(buffer));
            buffer = 0; bitcount = 0;
        }
    };

```



```

};

for (auto b : data) {
    const auto& code = codes[b];
    for (bool bit : code) flush_bit(bit);
}
// pad remaining bits
if (bitcount > 0) {
    out.put(static_cast<char>(buffer));
}
}

void Huffman::decompress(std::istream& in, std::ostream& out) {
    char magic[4];
    if (!in.read(magic, 4)) throw std::runtime_error("Bad header");
    if (std::memcmp(magic, "CMP1", 4) != 0) throw std::runtime_error("Not CMP1");
    int algo = in.get();
    if (algo != 0x01) throw std::runtime_error("Wrong algo id for Huffman");
    uint64_t orig_size = readU64(in);
    std::array<uint64_t, 256> freq{};
    for (int i=0; i<256; ++i) freq[i] = readU32(in);

    auto root = buildTree(freq);
    // Decode
    uint64_t produced = 0;
    Huffman::Node* node = root.get();

    int bytev;
    int bitpos = 8; // force initial reload
    uint8_t cur = 0;
    auto next_bit = [&]() -> int {
        if (bitpos == 8) {
            bytev = in.get();
            if (bytev == EOF) return -1;
            cur = static_cast<uint8_t>(bytev);
            bitpos = 0;
        }
        int bit = (cur >> bitpos) & 1;
        bitpos++;
        return bit;
    };

    while (produced < orig_size) {
        int bit = next_bit();
        if (bit < 0) break; // unexpected EOF
        node = bit == 0 ? node->left.get() : node->right.get();
    }
}

```

```

        if (!node) throw std::runtime_error("Corrupt bitstream (invalid traversal)");
        if (node->symbol >= 0) {
            out.put(static_cast<char>(uint8_t(node->symbol)));
            produced++;
            node = root.get();
        }
    }
    if (produced != orig_size) throw std::runtime_error("Size mismatch after Huffman
decode");
}

```

LZW.h

```

#pragma once
#include <cstdint>
#include <vector>
#include <unordered_map>
#include <string>
#include <istream>
#include <ostream>

// LZW implementation using 12-bit codes (max dictionary size 4096).
// File format when writing:
// [magic: 'CMP1'] [algo: 1 byte=0x02 for LZW]
// [orig_size: uint64 little-endian]
// [payload: stream of 12-bit codes packed into bytes, LSB-first]
class LZW {
public:
    static void compress(std::istream& in, std::ostream& out);
    static void decompress(std::istream& in, std::ostream& out);

private:
    static void writeU64(std::ostream& out, uint64_t v);
    static uint64_t readU64(std::istream& in);
};

```

LZW.cpp

```

#include "LZW.h"
#include <stdexcept>
#include <vector>
#include <cstring>

static void write_u8(std::ostream& out, uint8_t v){ out.put(static_cast<char>(v)); }
static int read_u8(std::istream& in){ int c=in.get(); return c; }

```

```

void LZW::writeU64(std::ostream& out, uint64_t v){
    for(int i=0;i<8;++i) out.put(static_cast<char>((v>>(8*i))&0xFF));
}

uint64_t LZW::readU64(std::istream& in){
    uint64_t v=0; for(int i=0;i<8;++i){ int c=in.get(); if(c==EOF) throw
std::runtime_error("Unexpected EOF"); v|=(uint64_t(uint8_t(c))<<(8*i)); }
    return v;
}

void LZW::compress(std::istream& in, std::ostream& out){
    std::vector<uint8_t> data((std::istreambuf_iterator<char>(in)),
std::istreambuf_iterator<char>());
    uint64_t orig_size = data.size();

    // Header
    out.write("CMP1",4);
    write_u8(out, 0x02);
    writeU64(out, orig_size);

    // Build initial dictionary for all single bytes
    const int MAX_BITS = 12;
    const int MAX_DICT_SIZE = 1<<MAX_BITS; // 4096
    std::unordered_map<std::string, int> dict;
    dict.reserve(5000);
    for (int i=0;i<256;++i) {
        std::string s(1, char(uint8_t(i)));
        dict[s] = i;
    }
    int next_code = 256;

    // Helper for 12-bit packing
    uint32_t bitbuf = 0;
    int bitcount = 0;
    auto emit_code = [&](int code){
        bitbuf |= (uint32_t(code) << bitcount);
        bitcount += 12;
        while (bitcount >= 8){
            write_u8(out, uint8_t(bitbuf & 0xFF));
            bitbuf >>= 8;
            bitcount -= 8;
        }
    };
    auto flush_codes = [&]() {
        if (bitcount > 0) {
            write_u8(out, uint8_t(bitbuf & 0xFF));
            bitbuf = 0; bitcount = 0;
        }
    };

```

```

    }
};

// LZW encoding
std::string w;
for (uint8_t k : data){
    std::string wk = w + char(k);
    if (dict.find(wk) != dict.end()) {
        w = std::move(wk);
    } else {
        emit_code(dict[w]);
        if (next_code < MAX_DICT_SIZE) {
            dict[wk] = next_code++;
        }
        w = std::string(1, char(k));
    }
}
if (!w.empty()) emit_code(dict[w]);
flush_codes();
}

void LZW::decompress(std::istream& in, std::ostream& out){
    char magic[4];
    if (!in.read(magic, 4)) throw std::runtime_error("Bad header");
    if (std::memcmp(magic, "CMP1", 4) != 0) throw std::runtime_error("Not CMP1");
    int algo = in.get();
    if (algo != 0x02) throw std::runtime_error("Wrong algo id for LZW");
    uint64_t orig_size = readU64(in);

    const int MAX_BITS = 12;
    const int MAX_DICT_SIZE = 1<<MAX_BITS; // 4096

    // Initialize dictionary
    std::vector<std::string> dict(MAX_DICT_SIZE);
    for (int i=0; i<256; ++i) dict[i] = std::string(1, char(uint8_t(i)));
    int next_code = 256;

    // 12-bit unpacking (LSB-first)
    uint32_t bitbuf = 0;
    int bitcount = 0;
    auto read_code = [&]() -> int {
        while (bitcount < 12){
            int c = in.get();
            if (c == EOF) {
                if (bitcount == 0) return -1;
                // trailing bits leftover is an error for this format
            }
            bitbuf = (bitbuf << 8) | c;
            bitcount += 8;
        }
        return (bitbuf >> (32 - bitcount)) & ((1 << bitcount) - 1);
    };

```

```

        return -2;
    }
    bitbuf |= (uint32_t(uint8_t(c)) << bitcount);
    bitcount += 8;
}
int code = int(bitbuf & 0xFFF);
bitbuf >>= 12;
bitcount -= 12;
return code;
};

int code = read_code();
if (code < 0) return;
std::string w = dict[code];
out.write(w.data(), w.size());
uint64_t produced = w.size();

int k;
while ((k = read_code()) >= 0) {
    std::string entry;
    if (k < next_code && !dict[k].empty()) {
        entry = dict[k];
    } else if (k == next_code) {
        // KwKwK case
        entry = w + w[0];
    } else {
        throw std::runtime_error("Corrupt LZW stream");
    }
    out.write(entry.data(), entry.size());
    produced += entry.size();

    if (next_code < MAX_DICT_SIZE) {
        dict[next_code++] = w + entry[0];
    }
    w = std::move(entry);
    if (produced >= orig_size) break;
}
if (produced != orig_size) {
    // We allow some containers to end exactly at size.
    if (produced < orig_size) throw std::runtime_error("Size mismatch after LZW decode");
}
}

```

main.cpp

```
#include <iostream>
#include <fstream>
#include <string>
#include "Huffman.h"
#include "LZW.h"

// Command-line frontend.
// Usage:
// compress_tool compress huffman input_file output_file
// compress_tool compress lzw input_file output_file
// compress_tool decompress huffman input_file output_file
// compress_tool decompress lzw input_file output_file
//
// Notes:
// - Output uses a simple container with a 'CMP1' magic + algo id.
// - Huffman stores a 256xuint32 frequency table for deterministic decode.
// - LZW uses 12-bit codes up to 4096 dictionary entries.
//
// Build (with CMake):
// mkdir build && cd build && cmake .. && cmake --build . -j
//
// Simple test (Linux/macOS):
// ./compress_tool compress huffman ../README.md out.cmp
// ./compress_tool decompress huffman out.cmp restored.md
//
// Error handling is basic to keep the demo compact.

int main(int argc, char** argv){
    if (argc != 5) {
        std::cerr << "Usage:\n"
            << " " << argv[0] << " compress huffman|lzw <in> <out>\n"
            << " " << argv[0] << " decompress huffman|lzw <in> <out>\n";
        return 1;
    }
    std::string mode = argv[1];
    std::string algo = argv[2];
    std::string inpath = argv[3];
    std::string outpath = argv[4];

    std::ifstream in(inpath, std::ios::binary);
    if (!in) { std::cerr << "Cannot open input file: " << inpath << "\n"; return 1; }
    std::ofstream out(outpath, std::ios::binary);
    if (!out) { std::cerr << "Cannot open output file: " << outpath << "\n"; return 1; }
```

```

try {
    if (mode == "compress") {
        if (algo == "huffman") Huffman::compress(in, out);
        else if (algo == "lzw") LZW::compress(in, out);
        else { std::cerr << "Unknown algorithm\n"; return 2; }
    } else if (mode == "decompress") {
        if (algo == "huffman") Huffman::decompress(in, out);
        else if (algo == "lzw") LZW::decompress(in, out);
        else { std::cerr << "Unknown algorithm\n"; return 2; }
    } else {
        std::cerr << "Unknown mode\n"; return 2;
    }
} catch (const std::exception& e) {
    std::cerr << "Error: " << e.what() << "\n";
    return 3;
}
return 0;
}

```

CMakeLists.txt

```

cmake_minimum_required(VERSION 3.10)
project(CompressionSuite CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(compress_tool
    main.cpp
    Huffman.cpp
    LZW.cpp
)

# For Windows static runtime (optional)
if (MSVC)
    target_compile_options(compress_tool PRIVATE /W4)
else()
    target_compile_options(compress_tool PRIVATE -Wall -Wextra -O2)
endif()

```